Rapport du projet “Analyse et Exploration d’un Réseau Social”

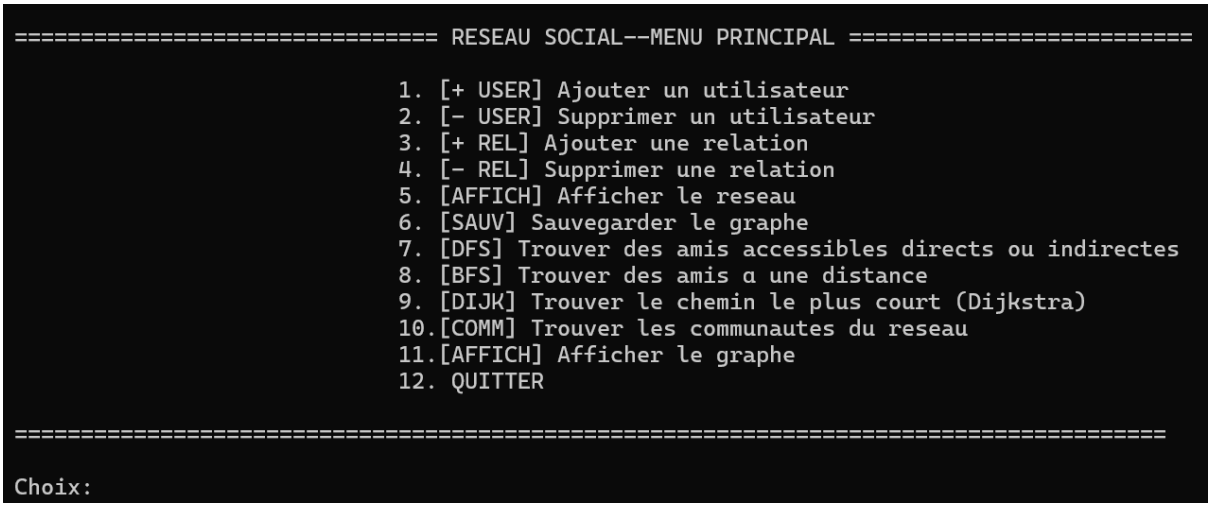
Syrine Guermazi
1ere LNBC 3

Analyse et Exploration d’un Réseau Social

Introduction:

Le code source de ce projet, écrit en C et organisé en plusieurs fichiers .c et .h, implémente un graphe pondéré pour modéliser un réseau social. Chaque module gère une responsabilité claire : main.c fait appel aux fonctions implémentées selon le choix de l’utilisateur, graph.h et graph.c gère les fonctions principales du réseau social (ajout/suppression d’un utilisateur/graphe, parcours du graphe, sauvegarde du graphe etc), ELTUSER.h, ELTUSER.c et ELTPRIM.h pour les fonctions concernant le TDA ELEMENT utilisateur (creation/destruction d’un utilisateur, affichage d’un utilisateur etc) et LSTPTR.h et LSTPTR.c se concentre sur les structures Node et adjList (création/destruction des structures).

Ce rapport présente brièvement l’objet et l’organisation de chaque composant, ainsi que leur rôle dans la construction, l’analyse et la visualisation du réseau.



Explication du travail:

Décomposition du projet par fichier :

main.c:

-Crée un graphe vide de taille 20
-Ajoute des utilisateurs et relations par défaut
-Présente une boucle itérative où l'utilisateur choisit parmis 12 options
-Detruit le graphe après la fermeture du programme (l'utilisateur entre 12.QUITTER)

graph.h / graph.c:

-Définition du TDA graphPondere
-Création, destruction du graphe
-Ajout/suppression d'utilisateurs et de relations
-Parcours (DFS, BFS), plus court chemin (Dijkstra)
-Chargement/sauvegarde dans un fichier graph.dot, affichage du réseau
-Détection de communautés

ELTUSER.h / ELTUSER.c:

-TDA ELEMENT (utilisateur) : champs id, nom, age
-Création (elementCreer), saisie (elementLire), affichage(elementAfficher), destruction (elementDetruire) etc…

ELTPRIM.h:

Signatures des opérations de base sur ELEMENT (primitives de copie, comparaison etc)

LSTPTR.h / LSTPTR.c:

-Définition du nœud structNode (Node) pour liste d'adjacence
-Création (creerNoeud), destruction(detruireNoeud)

examinons quelques fonctions intéressantes de ces fichiers:

1) int ajouterUtilisateur(graphePondere g, ELEMENT user):

```
Choix: 1

ID: 5
Nom: Amar
Age: 33

Utilisateur ajoute avec succes!
```

-Vérifie que le graphe et l'utilisateur ne sont pas NULL.
-Parcourt le tableau "utilisateurs[]" pour détecter un ID en double ; si trouvé, affiche un message et renvoie 0.
-Recherche ensuite la première case libre (NULL) dans "utilisateurs[]" et y stocke le pointeur "user", renvoyant 1 en cas de succès, ou un message d'erreur si le graphe est plein.

2) int supprimerUtilisateur(graphePondere g, int idUser):

```
Choix: 2
Choisir l'un des utilisateurs a supprimer:
U 0 Ahmed
U 1 Karim
U 2 Rania
U 3 Youssef
U 4 Samira
Entrer ID utilisateur a supprimer: 0
Utilisateur 0 supprime avec succes.
```

-Elle localise d'abord son indicedans "utilisateurs[]"
-Pour chaque autre sommet, elle utilise supprimerRelation pour enlever toutes les arêtes le reliant à lui.
-Elle libère sa propre liste d'adjacences et marque son entrée NULL.

3) int ajouterRelation(graphePondere g, int src, int dest, int poids):

```
Choix: 3
Entrer ID source: 0
Entrer ID destination: 4
Entrer poids (force relation): 3
Relation ajoutee avec succes!
```

-Elle insère une arête non orientée entre les indices src et dest.
-Après avoir vérifié que les deux sommets existent, elle crée deux nœuds: l'un pour src vers dest, l'autre pour dest vers src, qu'elle ajoute en tête de chaque liste d'adjacence.

4) int supprimerRelation(graphePondere g, int src, int dest):

```
Choix: 4
Entrer ID source: 0
Entrer ID destination: 4
Relation entre 0 et 4 supprimee avec succes.
```

-Vérifie que g n'est pas NULL et que src et dest sont dans les bornes [0,V–1], sinon affiche une erreur et renvoie 0.
-Parcourt la liste "adjList[src]"; dès qu'il trouve un nœud dont user->id==dest, il recolle le chaînage ("g->adjList[src]=p->next" ou "q->next=p->next") et libère le nœud.
-Répète symétriquement dans "adjList[dest]" pour supprimer l'arête inverse, garantissant la cohérence d'un graphe non orienté, puis renvoie 1.

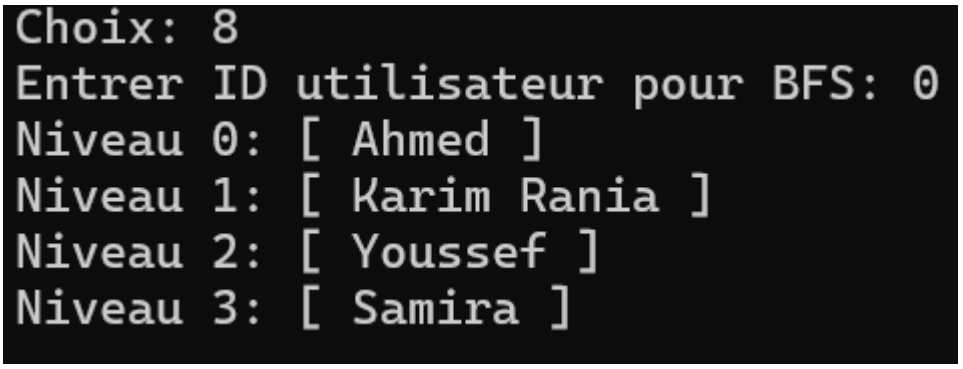
5) void dfs(graphePondere g, int idUser, int* visite):

```
Choix: 7
Entrer ID utilisateur pour DFS: 0
[Ahmed Karim Rania Youssef Samira]
```

-Alloue et initialise un tableau "visite[g->V]" à 0
-Lance la fonction récursive dfs pour parcourir en profondeur tous les voisins directs et indirects

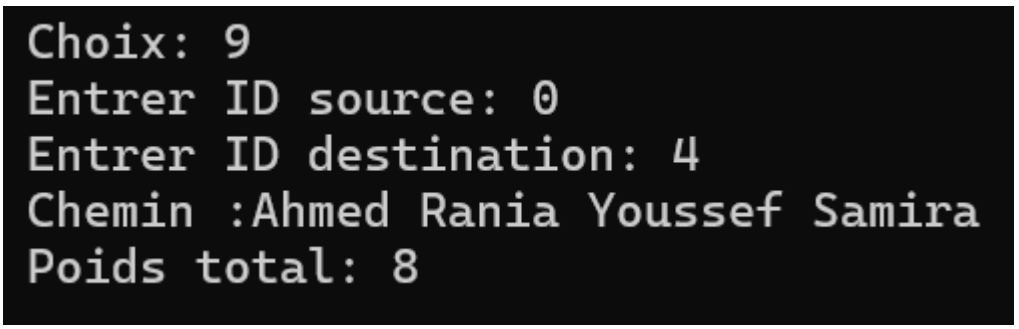
-Affiche en sortie la liste des noms d'utilisateurs accessibles du tableau "visite" depuis ce point de départ.

6) void bfs(graphePondere g, int idUser):



-Utilise une file FIFO pour explorer d'abord tous les voisins à distance 1, puis distance 2, etc…
-Remplit un tableau "niveau[g->V]" où "niveau[i]" est la distance (en nombre d'arêtes) entre idUser et i
-Affiche les utilisateurs regroupés par niveau, révélant les « cercles » d'amitié.

7) void dijkstra(graphePondere g, int src_id, int dest_id):



-Implémente l'algorithme de Dijkstra pour trouver le chemin de poids minimal (somme des forces d'amitié) entre src_id et dest_id
-Maintient trois tableaux dynamiques :
    -int* dist: pour la distance minimale courante,
    -int* visiter: pour les sommets traités,
    -int* prec: pour reconstruire le chemin.
-À chaque itération, sélectionne le sommet non visité de plus petite distance(dist[]), met à jour les voisins, et termine en affichant le chemin inversé et le poids total.

8) void trouver_communautes(graphePondere g):

```
Choix: 10
Communaute 1: [ Ahmed Karim Rania Youssef Samira ]
Communaute 2: [ Hichem ]
```

-Parcourt tout le graphe pour identifier les communautés en utilisant un algorithme de parcours en largeur (BFS) pour détecter ces communautés
-Affiche chaque communauté comme un groupe d'utilisateurs connectés entre eux.

9) void chargerReseau(graphePondere g):

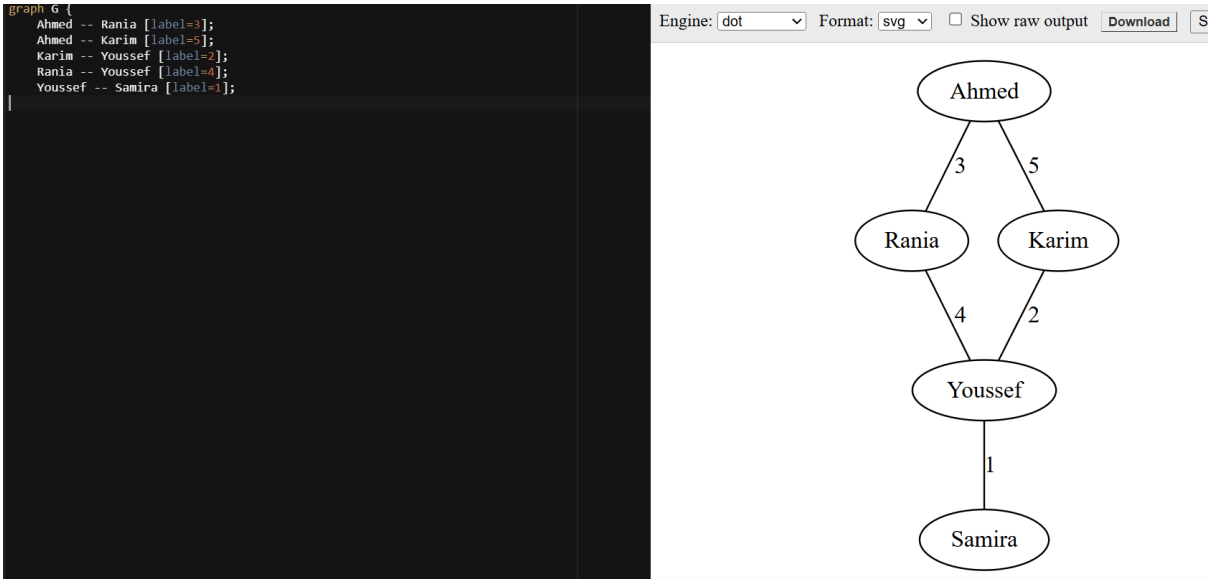
```
U 0 Ahmed 20
U 1 Karim 20
U 2 Rania 20
U 3 Youssef 20
U 4 Samira 20
U 5 Hichem 33
R 0 2 3 # Ahmed Rania
R 0 1 5 # Ahmed Karim
R 1 3 2 # Karim Youssef
R 2 3 4 # Rania Youssef
R 3 4 1 # Youssef Samira
```

-Affiche tous les utilisateurs et relations du graphe au format: "U id nom age" et "R src dest poids # nom_src nom_dest"
-Utilise une matrice pour éviter l'affichage en double des relations non orientées.

10) void sauvegarderGraphe(graphePondere g, const char* filename) (+BONUS):

```
graph G {
    Ahmed -- Rania [label=3];
    Ahmed -- Karim [label=5];
    Karim -- Youssef [label=2];
    Rania -- Youssef [label=4];
    Youssef -- Samira [label=1];
}
```

-Sauvegarde l'état du graphe dans un fichier graph.dot.
-Elle écrit chaque arête une seule fois (en évitant les doublons dans un graphe non orienté) et remplace les identifiants des utilisateurs par leurs noms pour générer un fichier lisible et exploitable par des outils comme Graphviz pour la visualisation graphique du réseau.



https://is.gd/z2yr3V

Conclusion:

Ce projet démontre avec succès l'implémentation d'un réseau social sous forme de graphe pondéré, permettant la gestion d'utilisateurs et de leurs relations. Il fournit un exemple pratique d'utilisation des structures de données et d'algorithmes de parcours de graphe pour modéliser des interactions sociales.